Week 6 (Pandas)

pd.Series → one column
pd.DataFrame → a table built from pd.Series

1. making a Series with a list:

```
import pandas as pd

values = [10, 20, 30, 40]
s = pd.Series(values)
print(s)
```

2. DataFrames

Can be built using a dict:

```
df = pd.DataFrame({
        'label': ["R-001", "R-002"],
        'value': [87, 100, 95]
    })
```

a dataframe can also be read from a csv file:

pd.read_csv("example.csv")

3. Adding a new column to a DataFrame

df["new_col"] = "test"

↳ makes a new column with default values as 'test'

4. print types of columns using: print(df.dtypes)

5. print only certain columns:

print(df[["col1", "col2"]])

↳ just shows col1 and col2

6. indexing

index using integers: df.iloc()
index using labels: df.loc()

7. Boolean filtering:

over_limit = servers[servers['p'] >= 10
print(over_limit)

8. .groupby()

splits the rows into groups with similar value for a specific column

servers.groupby('region')

↳ groups ~~servers by~~ rows by region

9. .agg()

aggregate function to print one summary at once:

```
print(grouped['percent'].agg(['min', 'max', 'mean']))
```

10. value_counts()

counts the number of occurrences for each value:

```
print(servers["used"].value_counts())
```

11. Sorting

sort_values() → to sort values by one or more columns.

```
print(s.sort_values(['region', 'percent']))
```

12. .apply()

runs a function on every value in a column

~~double~~

```
def double(x):
    return x*2
```

```
servers['doubled'] = servers['x'].apply(doub
```

13. lambda

↳ inline functions:

```
double = lambda x: x*2
```